

Neural Networks

APRENDIZAGEM PROFUNDA
LICENCIATURA EM ENGENHARIA INFORMÁTICA
ISEP/IPP

ANA MARIA MADUREIRA
2025/2026

43

43

Artificial Neural Networks (ANNs)

- **Artificial Neural Networks (ANNs)** are a type of machine learning algorithm that are inspired by the structure and function of the human brain. They consist of a large number of interconnected artificial neurons, which are organized into layers.
- An ANN is composed of an input layer, one or more hidden layers and an output layer. The input layer receives the input data, and the output layer produces the output. The hidden layers are used to extract features from the input data and to make predictions or decisions. The neurons in the network are connected to each other, and the information flows through the network in a feedforward manner, from the input layer to the output layer.
- The neurons in an ANN are modeled after the biological neurons in the brain, they take input from other neurons, process it and generate output. Each neuron receives input from multiple other neurons, and the output of each neuron is connected to one or more other neurons. The strength of the connection between neurons is represented by a weight, which can be adjusted during the training process.
- ANNs can be trained using supervised learning algorithms, such as **backpropagation**, to learn the relationship between the input and output data. The training process involves adjusting the weights and biases of the neurons in the network, so that the output of the network matches the desired output.
- Artificial Neural Networks (ANN) have been successfully applied in a wide range of tasks, such as image classification, speech recognition, natural language processing, and control systems. ANNs have also been used in fields such as finance, medicine, and marketing, due to their ability to model complex, non-linear relationships between input and output variables.

44

44

NEURAL NETWORKS

- **Artificial Neural Networks** have a surprising number of characteristics observed in the human cognitive process: learning by experience; generalization from examples and abstraction of the essential characteristics of information that contains irrelevant facts.
- Learning can be defined as the ability to perform new tasks that could not have been done before, or to improve previous performance as a result of changes produced by the learning process.
- **Artificial Neural Networks** can modify their behavior in response to stimuli produced by the environment: regulating the intensity of the connection between the processing units; adapting the synapse weights; recognizing the information presented to their neurons.
- To build a neuronal network it is necessary to consider the following main steps:
 - Define the number and type of neurons;
 - Define how they connect (topology);
 - Start the connection weights;
 - Learn which weights are appropriate for the problem (learning)

45

45

Neural Network - Historic Perspective

1943 - McCulloch and Pitts proposed the model of an artificial neuron, also called the McCulloch-Pitts model
 1948 - Wiener published the book "Cybernetics", in 1961, where aspects of learning and self-organization were discussed
 1949 - Hebb published the book "The Organization of Behavior", in which Hebb's Rule was proposed
 1958 - Rosenblatt introduced the simple one-layer network - Perceptron
 1969 - Minsky and Paper published the book "Perceptrons" in which they demonstrated the limitations of a single layer network. The area then had a period of stagnation
 1982 - Hopfield published several works dedicated to the Hopfield Recurring Networks
 1982 - Kohonen developed the self-organizing maps with his name
 1986 - The backpropagation algorithm for multi-layer networks was rediscovered, since then there has been a notable growth in the area
 1985 - Boltzmann machines were proposed by Ackley, Hinton and Sejnowski
 1989 - Yann LeCun proposed convolutional neural networks (CNN), initially advocated and developed by Fukushima in 1980
 1992 - Neal moves forward with Bayesian networks
 1995 - Recurring long memory networks were presented by Jurgen Schmidhuber
 2006 - The appearance of Deep Neural Networks launched a new interest in the area with numerous applications
 2014 - Adversarial networks appeared by GoodFellow

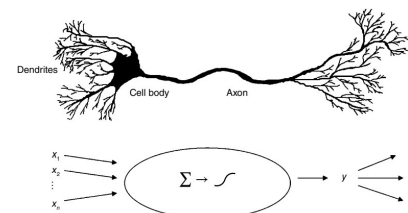
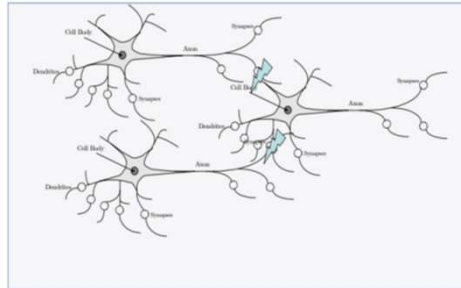
46

46

NEURAL NETWORKS

Neural networks represent an attempt at a very basic level to imitate the type of nonlinear learning that occurs in the networks of neurons found in nature, such as the human brain

NN models are inspired by biological neuron structures and computations.



Real neuron versus artificial neuron model

47

NEURAL NETWORKS

Most networks consist of three layers: an input layer, a hidden layer, and an output layer.

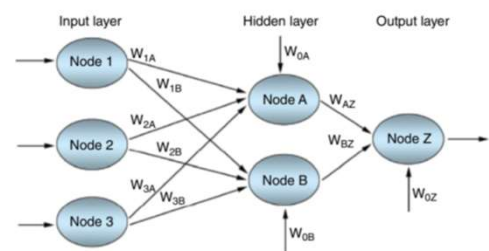
- There may be more than one hidden layer, although most networks contain only one, which is sufficient for most purposes.

The number of hidden layers and the number of nodes in each hidden layer are both configurable by the analyst. How many nodes should one have in the completely connected network of artificial neurons or nodes.

How many nodes should one have in the hidden layer?

Having more nodes in the hidden layer increases the power and flexibility of the network for identifying complex patterns – so, might be tempted to have a large number of nodes in the hidden layer.

- an overly large hidden layer leads to overfitting, memorizing the training set at the expense of generalizability to the validation set
- if overfitting is occurring, may consider to reduce the number of nodes in the hidden layer.
- if the training accuracy is unacceptably low, may consider increasing the number of nodes in the hidden layer.



A neural network consists of a layered, feedforward, completely connected network of artificial neurons or nodes:

- The feedforward nature of the network restricts the network to a single direction of flow and does not allow looping or cycling.

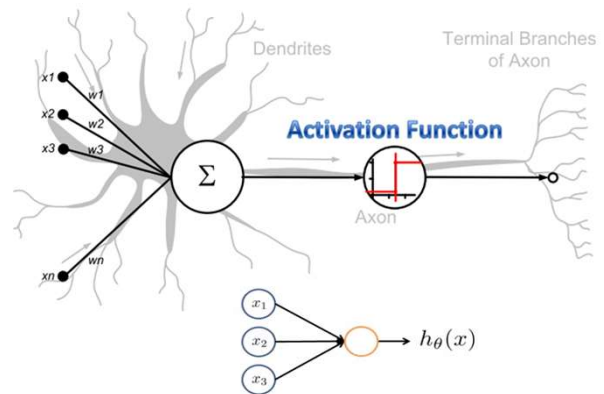
48

48

NEURAL NETWORKS – Neuron Model

• An artificial neuron (similar to the biological) has the function of performing a simple operation that consists of receiving signals from the input connections and calculate a new output value that is and sent out. Two phases can be identified:

- Calculate the weighted sum (**combination function**) of the input values;
- Calculate the neuron activation value using a function - the **activation function** or **transfer function**, which has an input argument - the value calculated in the previous step.

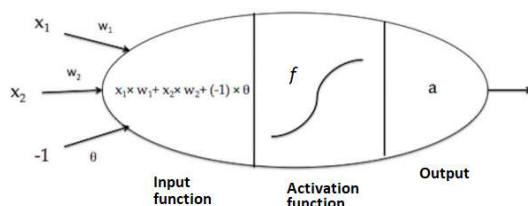


Model of artificial neuron (unit, code)

49

49

Artificial Neuron - mathematical model



f : activation function (or transfer function)
 Σ : input function or combination function
 w : input weight
 θ : bias or offset
 x_1, x_2 : inputs

- 2 inputs x_1, x_2 are visible with 2 associated weights w_1, w_2 and one input special value -1 , whose weight is called **offset** or **bias** and whose weight it is usually represented by the letter b or the Greek letter θ .
- The **combination function** is the weighted sum of these inputs is carried out in the first step of the operation of the neuron resulting in $\Sigma = x_1 \times w_1 + x_2 \times w_2 + (-1) \times \theta$.
- the **activation function f** , has as objective to determine whether the previously calculated weighted sum is sufficient to activate the neuron. the most common functions are the step function (sometimes also referred to as a signal function when the lower value is -1 instead of 0), the linear function and the sigmoid function.

50

50

Neural Network

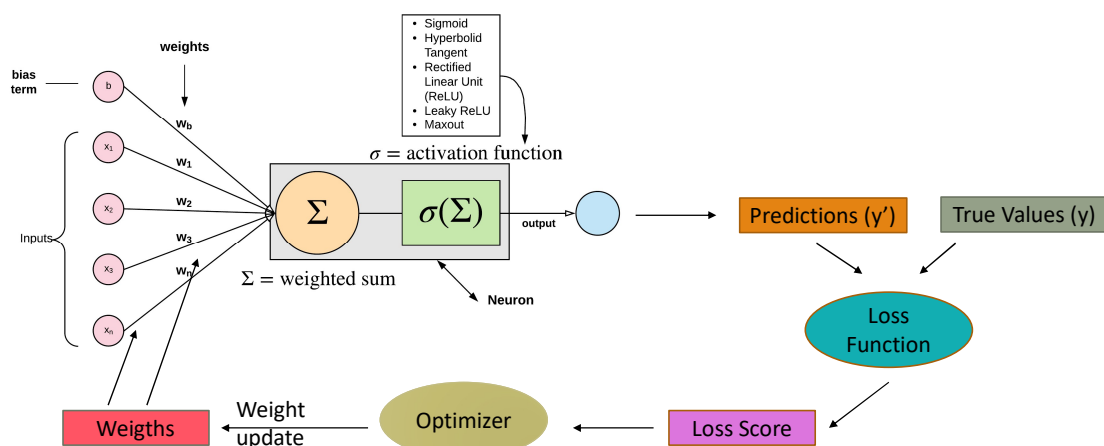
A neural network is a massively parallel distributed processor made up of simple processing units that has a natural propensity for storing experiential knowledge and making it available for use. It imitates the brain in two respects:

1. **Knowledge** is acquired by the network from its environment through a **learning process**.
2. Interneuron connection strengths, known as **synaptic weights**, are used to store the acquired knowledge.



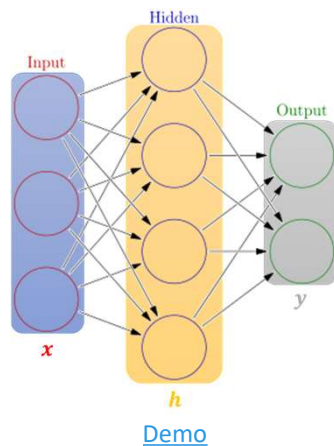
51

Neural Network



52

Neural Network



$$h = \sigma(W_1 x + b_1)$$

$$y = \sigma(W_2 h + b_2)$$

Weights

Activation functions

How do we train?

4 + 2 = 6 neurons (not counting inputs)

[3 x 4] + [4 x 2] = 20 weights

4 + 2 = 6 biases

26 learnable parameters

53

53

Importance & Significance

Hyperparameters are fundamental parameters that determine the architecture, behavior, and performance of neural networks.

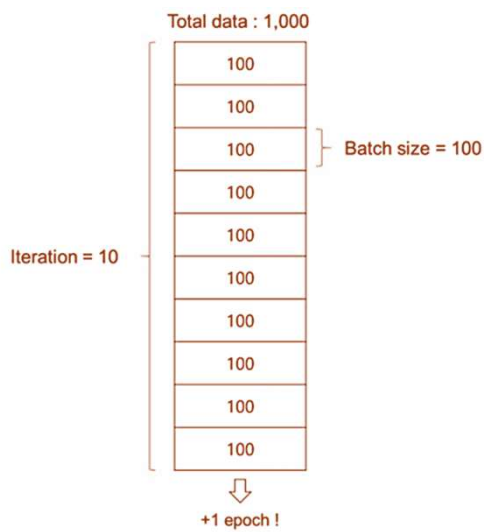
They play a pivotal role in shaping the learning process and optimizing model performance.

Proper selection and tuning of hyperparameters are essential for training deep learning models effectively.

Hyperparameters control various aspects such as network architecture, optimization algorithm behavior, and regularization techniques.

They directly influence the convergence, generalization, and overall efficacy of deep learning models.

54



Batch Size & Epochs

55

Batch Size



Definition

Batch size refers to the number of training examples utilized in one iteration of the gradient descent algorithm during training.



Effect on Networks

Batch size affects the efficiency of training and the stability of the learning process.

Larger batch sizes can lead to faster convergence but may require more memory and computational resources.

Smaller batch sizes may result in more noisy updates but can help the model generalize better.



Applicability & Insights

Large batch sizes are suitable for training on powerful hardware with ample memory, speeding up convergence in large datasets.

Small batch sizes are useful in scenarios where memory is limited, enabling training on resource-constrained environments or when working with streaming data.

56

Max Iterations/Epochs



Definition

Max iterations or epochs determine the maximum number of passes through the entire training dataset during the training process.



Effect on Networks

Max iterations/epochs control the duration of the training process and help prevent overfitting.

Early stopping, a technique based on monitoring performance on a validation dataset, can terminate training when performance starts to degrade.



Applicability & Insights

In scenarios where computational resources are limited, setting a maximum number of iterations/epochs helps ensure that training does not run indefinitely.

Early stopping is particularly useful in preventing overfitting and improving generalization in deep learning models, especially when training on large datasets with complex architectures.

57

Learning Rate

Definition

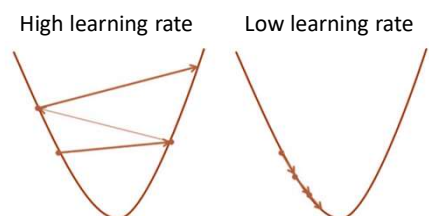
- Learning rate refers to the size of the steps taken during the optimization process to update the model parameters.

Effect on Networks

- Learning rate directly impacts the speed and stability of the training process.
- A high learning rate may cause the optimization algorithm to overshoot or oscillate around the minimum, leading to instability.
- A low learning rate may result in slow convergence or getting stuck in local minima.

Applicability & Insights

- Adaptive learning rate techniques (AdaGrad, RMSProp, and Adam, etc.) dynamically adjust the learning rate during training, improving convergence and stability.
- Learning rate schedules, such as exponential decay or step decay, gradually decrease the learning rate over time, helping to fine-tune the optimization process.



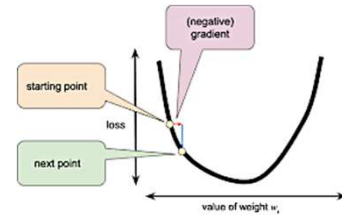
58

Gradients

Gradients are like arrows (mathematical vectors from partial derivatives) that show how much and in which direction the loss function changes when a parameter in the model is adjusted.

They are computed during optimization algorithms like gradient descent.

Gradients tell the direction and magnitude of the steepest ascent or descent in the loss landscape, helping to find the best parameters for the model



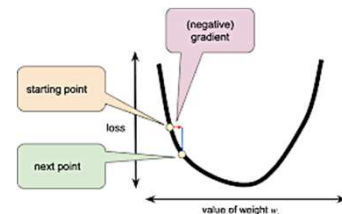
59

Gradient Descent

An optimization algorithm commonly used in machine learning and deep learning.

Minimizes the loss function of a model by adjusting its parameters (e.g., weights and biases).

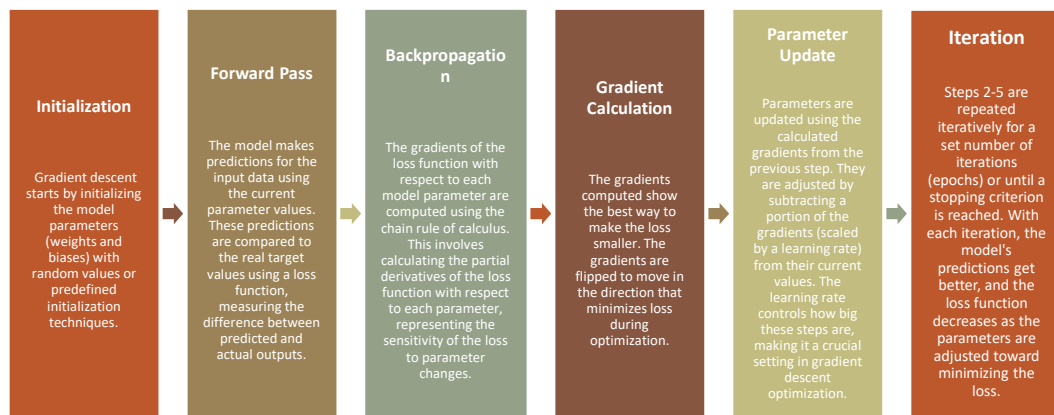
The goal is to iteratively update the model parameters in the direction of the steepest descent of the loss function, with the aim to reach a local minimum (ideally, the global minimum of the loss function)



60

Gradients & Gradient Descent

How do they work?



61

Hyperparameters in Neural Networks

Learning rate: This hyperparameter controls the step size taken by the optimizer during each iteration of training. Too small a learning rate can result in slow convergence, while too large a learning rate can lead to instability and divergence.

Epochs: This hyperparameter represents the number of times the entire training dataset is passed through the model during training. Increasing the number of epochs can improve the model's performance but may lead to overfitting if not done carefully.

Number of layers: This hyperparameter determines the depth of the model, which can have a significant impact on its complexity and learning ability.

Number of nodes per layer: This hyperparameter determines the width of the model, influencing its capacity to represent complex relationships in the data.

Architecture: This hyperparameter determines the overall structure of the neural network, including the number of layers, the number of neurons per layer, and the connections between layers. The optimal architecture depends on the complexity of the task and the size of the dataset.

Activation function: This hyperparameter introduces non-linearity into the model, allowing it to learn complex decision boundaries. Common activation functions include sigmoid, tanh, and Rectified Linear Unit (ReLU).

62

What is neural network optimization?

Neural network optimization:

- is the process of **finding the parameters that minimize the loss function**.
- is a specific type of optimization that deals with finding the optimal parameters of a neural network model, such as the **weights and biases that connect the neurons**.

A neural network model:

- is a function that maps some input data to some output data, by passing it through multiple layers of nonlinear transformations.
- can learn from data by adjusting its parameters to minimize a loss function, which measures the difference between the model's output and the desired output.

63

In machine learning, **optimizers** and **loss functions** are two fundamental components that help improve a model's performance.

- A **loss function** evaluates a model's effectiveness by computing the difference between expected and actual outputs. Common loss functions include log loss, hinge loss and mean square loss.
- An **optimizer** improves the model by adjusting its parameters (weights and biases) to minimize the loss function value. Examples include RMSProp, ADAM and SGD (Stochastic Gradient Descent).

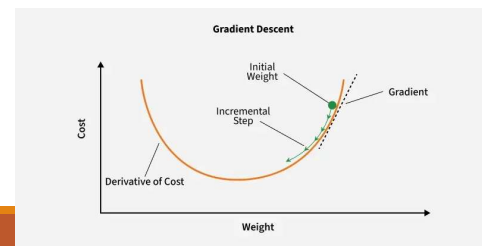
The **optimizer's role is to find the best combination of weights and biases that leads to the most accurate predictions**.

64

Gradient Descent

Gradient Descent is an **optimisation** algorithm used to minimize a model's error by iteratively adjusting its parameters. By moving step-by-step in the direction of the steepest decrease in the loss function, it helps machine learning models learn the best possible weights for accurate predictions.

- Essential for training neural networks and regression models
- Works by updating parameters based on calculated gradients
- Variants include **Batch**, **Stochastic** and **Mini-Batch Gradient Descent**



65

Neural Network optimization means updating the weights, biases of the model in order to minimize the Objective Function.

Key Steps in Gradient Descent

1. Initialize parameters: Randomly initialize the model parameters.
2. Compute the gradient: Calculate the gradient (derivative) of the loss function with respect to the parameters.
3. Update parameters: Adjust the parameters by moving in the opposite direction of the gradient, scaled by the learning rate.

Gradient Descent is one of the most well known algorithm. There are other variants / advanced techniques like **Stochastic Gradient Descent**, **Adam** that are widely used.

66

The Best Optimization Algorithm

Optimizers

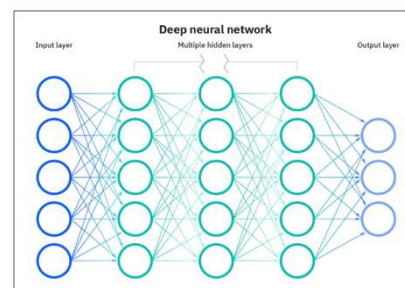
Each optimizer has its own strengths and weaknesses. The choice of optimizer depends on the **specific problem**, **dataset characteristics** and the **computational resources available**. Adam is often the default choice due to its robust performance, but each situation may call for a different optimizer to achieve optimal results.

Optimizer	Advantages	Disadvantages
SGD	Simple, easy to implement	Slow convergence, requires tuning
Mini-Batch SGD	Faster than SGD	Computationally expensive, stuck in local minima
SGD with Momentum	Faster convergence, reduces noise	Requires careful tuning of β
AdaGrad	Adaptive learning rates	Decays too fast, slow convergence
RMSProp	Prevents fast decay of learning rates	Computationally expensive
Adam	Fast, combines momentum and RMSProp	Memory-intensive, computationally expensive

67

What's the difference between deep learning and neural networks?

- **“deep”** in deep learning refers to the depth of layers in a neural network.
- Most deep neural networks are feed-forward, meaning they only flow in one direction from input to output.
- However, you can also train your model through **back-propagation**, meaning moving in the opposite direction, from output to input.
- **Back-propagation** allows us to calculate and attribute the error associated with each neuron, allowing us to adjust and fit the algorithm appropriately.



68

68

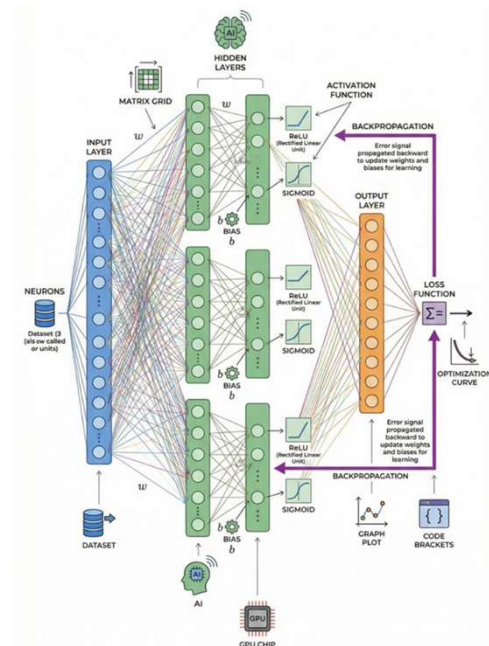
Deep Learning

Input → Hidden Layers → Activation →
Output → Loss → Backpropagation

Where:

- ✓ Weights learn
- ✓ Bias adjusts
- ✓ Error flows backward
- ✓ Model improves

From data to decision — optimized on GPU.

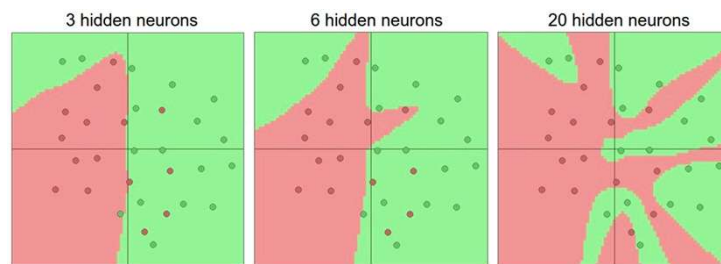


69

Activation functions

- Non-linearities needed to learn complex (non-linear) representations of data, otherwise the NN would be just a linear function

$$W_1 W_2 x = Wx$$



http://cs231n.github.io/assets/nn1/layer_sizes.jpeg

- More layers and neurons can approximate more complex functions

Full list: https://en.wikipedia.org/wiki/Activation_function

70

70

Overfitting vs Underfitting vs Generalization

Generalization

- usually refers to the ability of an algorithm to be effective across a range of inputs and applications.
- **refers to the model's ability to react to new data.**
- a ML algorithm must generalize from training data to help make accurate predictions while using the model.

Overfitting

- when the algorithm is not able to learn a model that represents the concepts under training examples. Is not able to generalize.
- refers to a model that models the training data too well.
- happens when a model learns the detail and noise in the training data to the extent that it negatively impacts the performance of the model on new data
- **good performance on the training data, poor generalization to other data**

Underfitting

- a failure to learn the relationships in the training data
- an underfitted model results in problematic or erroneous outcomes on new data, or data that it wasn't trained on, and often performs poorly even on training data.
- **Underfitting refers to a model that can neither model the training data nor generalize to new data.**
- Poor performance on the training data and poor generalization to other data
- Underfitting is often not discussed as it is easy to detect given a good performance metric. The solution is to move on and try another alternate ML algorithm.

Overfitting and underfitting cause poor generalization on the test set

71

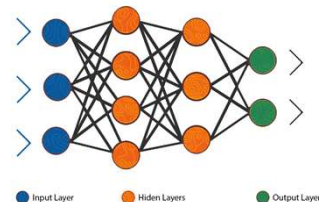
71

DIFFERENT TYPES OF NEURAL NETWORKS IN DEEP LEARNING

- Important types of neural networks that form the basis for most pre-trained models in deep learning:

- Artificial Neural Networks (ANN)
- Convolution Neural Networks (CNN)
- Recurrent Neural Networks (RNN)
- Generative Adversarial Networks (GAN)
- Self-Organizing Maps (SOM)

A single perceptron (or neuron) can be imagined as a Logistic Regression. Artificial Neural Network, or ANN, is a group of multiple perceptrons/ neurons at each layer. ANN is also known as a Feed-Forward Neural network because inputs are processed only in the forward direction



ANN consists of 3 layers – **Input, Hidden and Output**. The input layer accepts the inputs, the hidden layer processes the inputs, and the output layer produces the result. Essentially, each layer tries to learn certain weights.

72

72

Neural Network Topologies

There are several topologies of artificial neural networks with specific characteristics:

- networks have only forward connections (**feed-forward networks**)
- if there are feedback loops (**recurrent networks**).
- have a supervisor or teacher, that is, if they are supervised or if they have autonomous learning algorithms(**self-organized**).

73

73

Neural Networks Topologies

There are many different types of neural networks, each with its own strengths and weaknesses, and each suited for different types of tasks. **Some of the most commonly used types of neural networks:**

- **Feedforward neural networks (FNN):** Also known as multi-layer perceptrons (MLP), these are the most basic type of neural networks. They consist of an input layer, one or more hidden layers, and an output layer. The neurons in one layer are connected to the neurons in the next layer, and the information flows in one direction, from the input layer to the output layer. FNNs are used for tasks such as image classification, speech recognition, and language modeling.
- **Convolutional neural networks (CNN):** These are neural networks that are specifically designed for image and video processing tasks. CNNs consist of multiple layers of convolutional and pooling layers, which are designed to extract features from images and reduce their dimensionality. CNNs are used for tasks such as image classification, object detection, and semantic segmentation.
- **Recurrent neural networks (RNN):** These are neural networks that are designed to process sequences of data, such as time series or natural language text. RNNs consist of multiple layers of neurons, each of which is connected to the previous layer in a time-dependent manner. This allows RNNs to maintain an internal state, which allows them to process sequences of data. RNNs are used for tasks such as speech recognition, language translation, and text generation.
- **Generative Adversarial Networks (GAN):** These are neural networks that consist of two parts: a generator and a discriminator. The generator is trained to generate new data samples that are similar to the training data, while the discriminator is trained to distinguish the generated samples from the real samples. GANs are used for tasks such as image synthesis, image-to-image translation and data augmentation.
- **Self-Organizing Maps (SOM):** These are neural networks that are used for unsupervised learning tasks, such as dimensionality reduction and clustering. SOMs consist of a grid of neurons, each of which is trained to represent a specific part of the input space. SOMs are used for tasks such as visualizing high-dimensional data, anomaly detection and feature extraction.

74

74

Feedforward neural networks (FNN)

Feedforward neural networks (FNN), also known as Multi-Layer Perceptrons (MLP), are the most basic type of neural networks. They consist of an input layer, one or more hidden layers, and an output layer. The neurons in one layer are connected to the neurons in the next layer, and the information flows in one direction, from the input layer to the output layer. The input layer receives the input data, and the output layer produces the output. The hidden layers are used to extract features from the input data and to make predictions or decisions.

FNNs are used for a wide range of tasks, such as image classification, speech recognition, and language modeling. They can be trained using supervised learning algorithms, such as backpropagation, to learn the relationship between the input and output data. The training process involves adjusting the weights and biases of the neurons in the network, so that the output of the network matches the desired output.

One of the main advantages of FNNs is their simplicity, which makes them easy to understand and implement. They can also be used to model non-linear relationships between the input and output data. However, their performance might be limited when the input data has high dimensionality or when the relationship between the input and output data is complex.

75

75

Applications

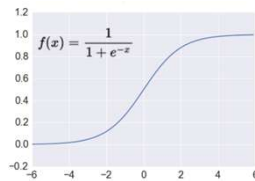
- **Health care:** With easier access to accelerated GPU and the availability of huge amounts of data, health care use cases have been a perfect fit for applying DL.
- **Autonomous vehicles:** Though self-driving cars is a risky field to automate, it has recently taken a turn towards becoming a reality. From recognizing a stop sign to seeing a pedestrian on the road, deep learning-based models are trained and tried under simulated environments to monitor progress.
- **e-commerce:** Product recommendations has been one of the most popular and profitable applications. With more personalized and accurate recommendations, customers are able to easily shop for the items they are looking for and are able to view all of the options that they can choose from. This also accelerates sales and thus, benefits sellers.
- **Personal assistant:** Thanks to advancements in the field of DL, having a personal assistant is as simple as buying a device like Alexa or Google Assistant. These smart assistants use DL in several aspects such as personalized voice and accent recognition, personalized recommendations, and text generation.

Stock market predictions and weather predictions are also popular fields in which deep learning has been helpful.

76

76

Activation FUNCTION: Sigmoid



<http://adilmoujahid.com/images/activation.png>

Takes a real-valued number and “squashes” it into range between 0 and 1.

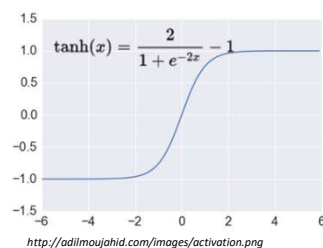
$$\mathbb{R}^n \rightarrow [0,1]$$

- + Nice interpretation as the **firing rate** of a neuron
 - 0 = not firing at all
 - 1 = fully firing
- Sigmoid neurons **saturate** and **kill gradients**, thus NN will barely learn
 - when the neuron’s activation are 0 or 1 (saturate)
 - gradient at these regions almost zero
 - almost no signal will flow to its weights
 - if initial weights are too large then most neurons would saturate

77

77

Activation: Tanh



<http://adilmoujahid.com/images/activation.png>

Takes a real-valued number and “squashes” it into range between -1 and 1.

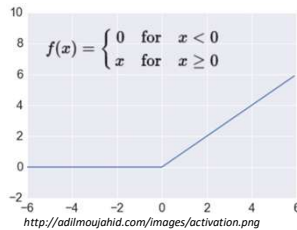
$$\mathbb{R}^n \rightarrow [-1,1]$$

- Like sigmoid, tanh neurons **saturate**
- Unlike sigmoid, output is **zero-centered**
- Tanh is a **scaled sigmoid**: $\tanh(x) = 2\text{sigm}(2x) - 1$

78

78

Activation Function: ReLU



Takes a real-valued number and thresholds it at zero

$$f(x) = \max(0, x)$$

$$R^n \rightarrow R_+^n$$

Most Deep Networks use ReLU :

- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- Trains much **faster**
 - accelerates the convergence of SGD
 - due to linear, non-saturating form
- Less expensive operations
 - compared to sigmoid/tanh (exponentials etc.)
 - implemented by simply thresholding a matrix at zero
- More **expressive**
- Prevents the **gradient vanishing problem**

79

79

Training and Optimization of Neural Networks

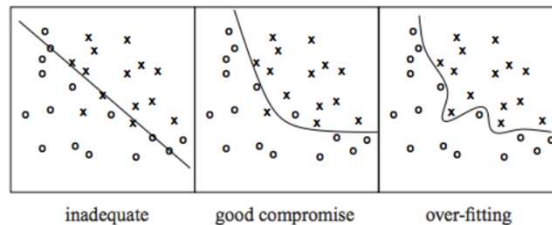
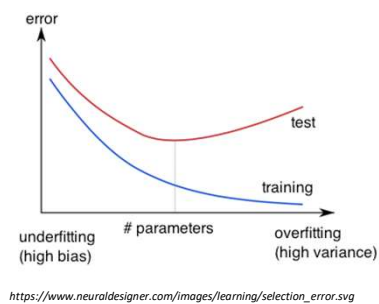
Focuses on finding the set of parameters W that minimize the loss function

- Gradient Descent
 - Stochastic Gradient Descent
- Backpropagation
- Generalization and Overfitting
 - Early Stopping
 - Weight Regularization
 - Weight Initialization
 - Drop-out
 - Data Augmentation
 - Batch Normalization

80

80

Overfitting



Learned hypothesis may **fit** the training data very well, even outliers (**noise**) but fail to **generalize** to new examples (test data)

81

81

Overfitting vs Underfitting vs Generalization

Overfitting

- when the algorithm is not able to learn a model that represents the concepts under training examples. Is not able to generalize.
- refers to a model that models the training data too well.
- happens when a model learns the detail and noise in the training data to the extent that it negatively impacts the performance of the model on new data
- **good performance on the training data, poor generalization to other data**

Underfitting

- a failure to learn the relationships in the training data
- an underfitted model results in problematic or erroneous outcomes on new data, or data that it wasn't trained on, and often performs poorly even on training data.
- **Underfitting refers to a model that can neither model the training data nor generalize to new data.**
- Poor performance on the training data and poor generalization to other data
- Underfitting is often not discussed as it is easy to detect given a good performance metric. The solution is to move on and try another alternate ML algorithm.

Generalization

- usually refers to the ability of an algorithm to be effective across a range of inputs and applications.
- **refers to the model's ability to react to new data.**
- a ML algorithm must generalize from training data to help make accurate predictions while using the model.

Overfitting and underfitting cause poor generalization on the test set

82

82

Neural Network Topologies

There are several topologies of artificial neural networks with specific characteristics:

- networks have only forward connections (**feed-forward networks**)
- if there are feedback loops (**recurrent networks**).
- have a supervisor or teacher, that is, if they are supervised or if they have autonomous learning algorithms(**self-organized**).

83

83

Feed-forward network

In general, a neuron, even with several inputs, is not enough to solve the majority of real problems. Several neurons are required to work in parallel, in what is usually called a layer of neurons, or even several layers of neurons constituting a multilayered architecture

A network with only forward or direct connections, with three layers of neurons, with the input layer ($[x_1, x_2]$). Each layer has its weight matrix w , the bias is not represented at this case.

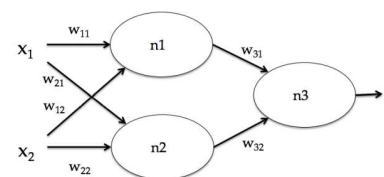
The output layer has only one neuron (n_3) and the intermediate layer, called a hidden layer, it consists of two neurons (n_1 and n_2), the various layers can have a different number of neurons.

When input layer and the output layer are all called the hidden layer (1st hidden layer, 2nd hidden layer, and so on from the input to the output). The outputs of the input layer constitute the inputs of the hidden layer, and so on, so that each layer can be interpreted as an isolated layer.

Faced with a problem to be solved with feed-forward neural networks, the number of neurons in the input and output layers are usually easy to define by the problem definition itself.

The number of neurons in each hidden layer and the number of hidden layers is more subjective. There are many heuristics to choose the structure of a neural network, there is no method that allows to choose exactly the best structure for a neural network.

If the network is too small, you may not be able to solve the proposed problem. While if the network is too large, despite being able to memorize the examples presented, it may prove unable to generalize beyond those examples - it may lose its most important capacity - the ability to learn (overfitting)



Model of a neuronal network with three layers of neurons

84

84

Index

- **Neural Networks**
- **Activation Functions**
- **Loss Functions**
- **Internal Parameters (weights & Bias)**

85

Activation Functions

Introduction to Activation Functions

Purpose and Importance in Deep Learning Models

Characteristics

Commonly Used Activation Functions, Strengths & Weaknesses:

- Sigmoid
- Tanh
- ReLU (Rectified Linear Unit)
- Softmax (for multi-class classification)

86

Introduction to Activation Functions

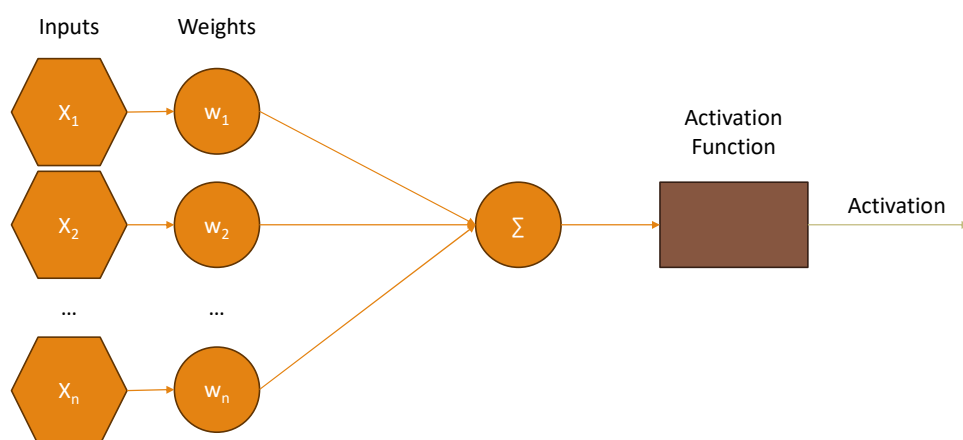
Activation functions are mathematical functions applied to the output of each neuron in a neural network.

They introduce non-linearity to the network, allowing it to learn complex patterns and relationships in data.

These functions determine whether a neuron should be activated or not, based on the weighted sum of its inputs.

87

Introduction to Activation Functions



88

Purpose and Importance in Deep Learning Models

Activation functions are crucial for the successful training of neural networks.

They enable networks to learn complex functions and make them capable of approximating any arbitrary function.

Without activation functions, neural networks would reduce to linear regression models, limiting their expressiveness and learning capacity.

89

Characteristics

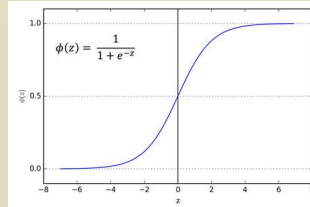
- **Non-linearity:** Activation functions introduce non-linearities, enabling neural networks to approximate non-linear functions.
- **Differentiability:** Activation functions should be differentiable to facilitate the optimization process during training using gradient-based methods.
- **Range:** Activation functions typically have a specific output range, which can affect the behavior of the network and the stability of training.

90

Sigmoid Activation Function

Defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Applicability

- Binary classification problems where outputs need to be interpreted as probabilities.
- Output layer for binary classification tasks.
- Logistic regression.

Strengths

- Smooth and bounded output between 0 and 1, mimicking probabilities.
- Useful in binary classification tasks where the output needs to be interpreted as probabilities. For **binary classification, it is also more efficient (compared to softmax)**.
- Well-suited for outputs that need to be in the range (0, 1).

Weaknesses

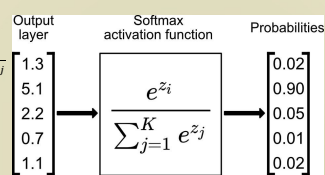
- Sigmoid saturates and kills gradients during backpropagation, known as the vanishing gradient problem.
- Outputs are not zero-centered, making optimization slower, especially in deep networks.
- Prone to the "vanishing gradient" problem, leading to slow convergence in deep networks.

91

Softmax Activation Function

Defined as:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$



Applicability

- Multi-class classification problems where the output needs to represent class probabilities.
- Output layer in neural networks for multi-class classification tasks.

Strengths

- Converts raw scores into probabilities that sum up to 1, making it suitable for multi-class classification.
- Outputs can be interpreted as class probabilities, aiding in decision-making.
- Used in the output layer of neural networks for multi-class classification tasks.

Weaknesses

- Sensitive to large input values, which can lead to numerical instability.
- Requires careful consideration of loss functions, especially for imbalanced datasets.

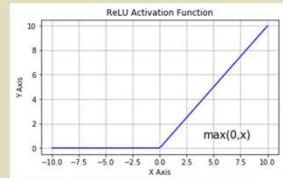
92

ReLU Activation Function

Defined as:

$$f(x) = \max(0, x)$$

"0" is a threshold that can be configured, but defaults to 0



Applicability

- Hidden layers in deep neural networks, especially in convolutional neural networks (CNNs).
- Rectification of feature maps in CNNs.

Strengths

- Simple and computationally efficient.
- Overcomes the vanishing gradient problem by avoiding saturation in positive regions.
- Fast convergence in deep networks due to linear, non-saturating activation.

Weaknesses

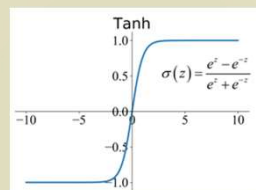
- Not zero-centered, which can lead to dead neurons (neurons that never activate).
- Prone to the "dying ReLU" problem where neurons can get stuck in the negative side and never recover.

93

Tanh Activation Function

Defined as:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Applicability

- Hidden layers in feedforward neural networks.
- Recurrent neural networks (RNNs) where zero-centered outputs are preferred.

Strengths

- Zero-centered outputs (-1 to 1), making optimization easier compared to sigmoid.
- Saturates less quickly than the sigmoid function, alleviating the vanishing gradient problem to some extent.
- Well-suited for outputs that need to be in the range (-1, 1).

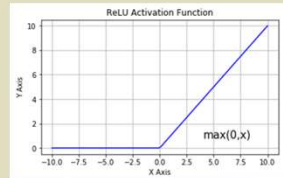
Weaknesses

- Still prone to the vanishing gradient problem, especially in deep networks.
- Outputs are not always interpretable as probabilities.

94

Linear Activation Function

Defined as:
 $f(x) = x$



Applicability

- Regression tasks where the output is expected to be a linear combination of input features.
- Simple linear models or shallow networks where non-linearity is not a requirement.
- Output layer in neural networks for regression tasks, predicting continuous values.

Strengths

- Direct mapping of input to output, preserving linearity.
- Suitable for tasks where a linear relationship between input and output is desired.
- Used in the output layer for regression tasks, providing continuous output values.

Weaknesses

- Lacks non-linearity, limiting the expressiveness of the network.
- Not effective for capturing complex patterns or non-linear relationships in the data.
- Prone to gradient vanishing or exploding problems in deep networks.

95

Loss Functions

Introduction to Loss Functions

Purpose and Importance in Deep Learning Models

Commonly Used Loss Functions, Strengths & Weaknesses:

○Regression

1. MSE(Mean Squared Error)
2. MAE(Mean Absolute Error)
3. Huber loss??

○Classification

1. Binary cross-entropy ~~Loss~~
2. Categorical cross-entropy ~~Loss~~
3. Sparse categorical cross-entropy

96

Introduction to Loss Functions

A **loss function** is a function that **compares** the target and predicted output values. The principal objective of neural networks is to minimize the loss function.

Measures how well the neural network models the training data. When training, we aim to minimize this loss between the predicted and true values.

The **hyperparameters** are adjusted to minimize the average loss.

97

Introduction to Loss Functions

Loss functions provide a **measure of how well the model is performing** on the given task. They **quantify the difference** between **predicted** outputs and **actual targets**.

They serve as the objective function during both model training and validation phases.

The goal of training is to minimize the loss, adjusting model parameters to improve performance.

Similarly, during validation, the loss is used to assess the model's generalization to unseen data.

The **hyperparameters** are adjusted to minimize the average loss.

98

Purpose and Importance in Deep Learning Models

- Loss functions are crucial for the successful training of neural networks.
- They are used to evaluate model performance.

99

Purpose and Importance in Deep Learning Models

Incorporating Task-specific Goals	Optimization	Evaluation	Regularization
• Different tasks require different loss functions tailored to their objectives. • Loss functions encode task-specific goals such as classification accuracy, regression error, or semantic similarity and these functions are used to evaluate model performance.	• Loss functions play a crucial role in optimization algorithms such as gradient descent. • By computing gradients with respect to the loss, the optimizer updates model parameters to minimize the loss and improve performance during both training and validation.	• Loss functions provide a quantitative metric for evaluating model performance, not only during training but also during validation. • They allow comparison between different models and parameter configurations, aiding in model selection and hyperparameter tuning.	• Certain loss functions incorporate regularization terms to penalize complex models, preventing overfitting and promoting generalization to unseen data during both training and validation.

100

Mean Squared Error (MSE)

Defined as:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

Applicability

- MSE finds the average of the squared differences between the target and the predicted outputs

Strengths

- Easy derivative
- Minor updates for small errors
- Easy to interpret.
- Always differential because of the square.
- Only one local minim.

Weaknesses

- Not robust to outliers

101

Mean Squared Error (MSE)

Defined as:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

MSE finds the average of the squared differences between the target and the predicted outputs

Applicability

- Regression tasks where the output is continuous and the goal is to minimize the average squared difference between predicted and actual values.
- Models where outliers are expected to be rare or can be effectively handled through data preprocessing or regularization techniques.

Strengths

- Easy derivative
- Minor updates for small errors
- Easy to interpret.
- Always differential because of the square.
- Only one local minim.

Weaknesses

- Sensitive to outliers, as large errors are penalized heavily.
- May prioritize minimizing large errors at the expense of smaller errors, leading to suboptimal performance in the presence of outliers.

102

Mean Absolute Error (MAE)

Defined as:

$$\text{MAE} = \sum_{i=0}^m |y^{(i)} - \hat{y}^{(i)}|$$

Applicability

Calculates the absolute differences between the predicted and true values.

Strengths

- Intuitive and easy.
- Robust to outliers.
- MAE can be shown to optimize according to the median of the errors.
- Error Unit Same as the output column.

Weaknesses

- No analytical solution
- **Gradient magnitude** is the same for all errors

103

Mean Absolute Error (MAE)

Defined as:

$$\text{MAE} = \sum_{i=0}^m |y^{(i)} - \hat{y}^{(i)}|$$

Calculates the absolute differences between the predicted and true values.

Applicability

- Regression tasks where outliers are common or influential, as MAE is less sensitive to outliers compared to MSE.
- Models where interpretability of error is important, as MAE provides a direct measure of average absolute error.

Strengths

- Intuitive and easy, as it directly represents the average magnitude of errors and error unit is the same as output's.
- Robust to outliers due to the use of absolute differences instead of squared differences.
- MAE can be shown to optimize according to the median of the errors.

Weaknesses

- **Gradient magnitude** is the same for all errors (i.e. sensitive to outliers, as large errors are penalized heavily).
- May prioritize minimizing large errors at the expense of smaller errors, leading to suboptimal performance in the presence of outliers.

104

Huber

Defined as:

$$\text{Huber Loss} = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2 \quad |y^{(i)} - \hat{y}^{(i)}| \leq \delta$$

$$\frac{1}{n} \sum_{i=1}^n \delta (|y^{(i)} - \hat{y}^{(i)}| - \frac{1}{2} \delta) \quad |y^{(i)} - \hat{y}^{(i)}| > \delta$$

Works as a hybrid of MSE and MAE losses

Applicability

- Regression tasks where outliers are present but their influence should be limited, as Huber loss provides a compromise between MSE and MAE.
- Models where a smooth and differentiable loss function is desired, facilitating optimization using gradient-based methods.

Strengths

- Combines the best properties of MSE and MAE, being robust to outliers while still providing smoothness and differentiability.
- Adapts to the magnitude of errors, using squared loss for small errors and absolute loss for large errors.

Weaknesses

- Requires tuning of the hyperparameter δ , which determines the threshold for switching between squared and absolute loss.
- Less commonly used compared to MSE and MAE, which may result in fewer available implementations or documentation.

105

Binary Cross-Entropy

Defined as:

$$\text{BinaryCrossentropy} = - \sum_{i=1}^C (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

y_i : True label (0 or 1)

$\hat{y}_i$: Probability of the model's prediction

Compares each of the predicted probabilities to the true class output which can be either 0 or 1

Applicability

- Binary classification tasks where the goal is to predict binary outcomes or probabilities associated with positive class labels.
- Models where a probabilistic interpretation of predictions is desired, as BCE loss directly optimizes the likelihood of the observed data under the model.

Strengths

- Specifically designed for binary classification tasks, where the output is binary (0 or 1) and represented as probabilities.
- Penalizes confident misclassifications more heavily (i.e. **exponentially**), contributing to sharper gradients and faster convergence during training.

Weaknesses

- Prone to saturation when predicted probabilities are close to 0 or 1, leading to vanishing gradients and slow learning.
- Sensitive to class imbalance, where rare or minority classes may receive less emphasis during training.
- Less interpretable and contains multiple local minima

106

Categorical Cross-Entropy

Defined as:

$$\text{CategoricalCrossentropy} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

C: Number of classes

y_i : Probability of the true label being in class i

$\hat{y}_i$: Probability of the model's prediction being in class i

Compares predicted probabilities across multiple classes to the true class output, where the true class is represented as a one-hot encoded vector.

Applicability

- Multi-class classification tasks where the output consists of multiple mutually exclusive classes.
- Models where predictions are represented as probabilities across multiple classes, allowing for probabilistic interpretation and uncertainty estimation.

Strengths

- Generalization of binary cross entropy to multi-class classification tasks, where the output consists of multiple classes and is represented as probabilities using one-hot encoding.
- Encourages the model to assign high probabilities to the correct class labels while penalizing confidently (i.e. exponentially) incorrect predictions.

Weaknesses

- Like binary cross entropy, prone to saturation and slow learning when predicted probabilities are close to 0 or 1.
- Requires one-hot encoding of class labels, which can be memory-intensive and computationally inefficient for large numbers of classes.
- Less interpretable and contains multiple local minima

107

Sparse Categorical Cross-Entropy

Defined as:

$$\text{SparseCategoricalCrossentropy} = - \sum_{i=1}^C \log(\hat{y}_i[y_i])$$

C: Number of classes

y_i : Integer representation of the true label (ranging from 0 to C)

$\hat{y}_i$: Probability of the model's prediction being in class i

Like Categorical Cross Entropy, but instead of using one-hot encoded vectors for true class labels, it uses integer indices.

Applicability

- Multi-class classification tasks where class labels are represented as integers instead of one-hot encoded vectors.
- Models where memory efficiency and simplicity of data preprocessing are prioritized, particularly for large datasets with many classes.

Strengths

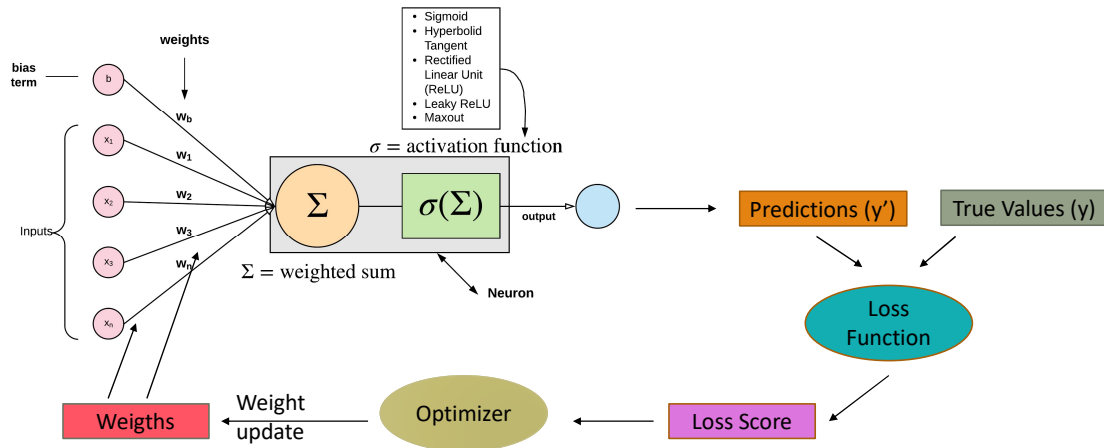
- Provides a more memory-efficient representation of class labels compared to one-hot encoding (i.e. categorical cross entropy), particularly for large numbers of classes.
- Simplifies the data preprocessing pipeline, as class labels can be represented directly as integers without the need for one-hot encoding.

Weaknesses

- Requires class labels to be represented as integers, which may limit the flexibility of the output representation compared to one-hot encoding.
- Prone to misclassification errors when class labels are noisy or ambiguous, as the loss function relies on exact matches between predicted and true class labels.
- Less interpretable and contains multiple local minima

108

Neural Network



109

Hyperparameters in Neural Networks

Learning rate: This hyperparameter controls the step size taken by the optimizer during each iteration of training. Too small a learning rate can result in slow convergence, while too large a learning rate can lead to instability and divergence.

Epochs: This hyperparameter represents the number of times the entire training dataset is passed through the model during training. Increasing the number of epochs can improve the model's performance but may lead to overfitting if not done carefully.

Number of layers: This hyperparameter determines the depth of the model, which can have a significant impact on its complexity and learning ability.

Number of nodes per layer: This hyperparameter determines the width of the model, influencing its capacity to represent complex relationships in the data.

Architecture: This hyperparameter determines the overall structure of the neural network, including the number of layers, the number of neurons per layer, and the connections between layers. The optimal architecture depends on the complexity of the task and the size of the dataset.

Activation function: This hyperparameter introduces non-linearity into the model, allowing it to learn complex decision boundaries. Common activation functions include sigmoid, tanh, and Rectified Linear Unit (ReLU).

110

Overfitting vs Underfitting vs Generalization

Generalization

- usually refers to the ability of an algorithm to be effective across a range of inputs and applications.
- **refers to the model's ability to react to new data.**
- a ML algorithm must generalize from training data to help make accurate predictions while using the model.

Overfitting

- when the algorithm is not able to learn a model that represents the concepts under training examples. Is not able to generalize.
- refers to a model that models the training data too well.
- happens when a model learns the detail and noise in the training data to the extent that it negatively impacts the performance of the model on new data
- **good performance on the training data, poor generalization to other data**

Underfitting

- a failure to learn the relationships in the training data
- an underfitted model results in problematic or erroneous outcomes on new data, or data that it wasn't trained on, and often performs poorly even on training data.
- **Underfitting refers to a model that can neither model the training data nor generalize to new data.**
- Poor performance on the training data and poor generalization to other data
- Underfitting is often not discussed as it is easy to detect given a good performance metric. The solution is to move on and try another alternate ML algorithm.

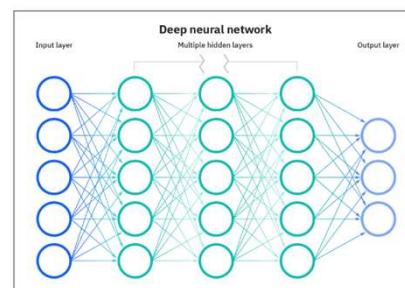
Overfitting and underfitting cause poor generalization on the test set

111

111

What's the difference between deep learning and neural networks?

- **"deep"** in deep learning refers to the depth of layers in a neural network.
- Most deep neural networks are feed-forward, meaning they only flow in one direction from input to output.
- However, you can also train your model through **back-propagation**, meaning moving in the opposite direction, from output to input.
- **Back-propagation** allows us to calculate and attribute the error associated with each neuron, allowing us to adjust and fit the algorithm appropriately.



112

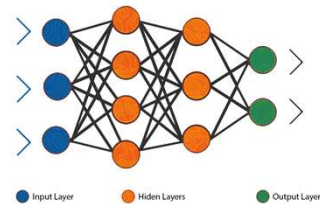
112

DIFFERENT TYPES OF NEURAL NETWORKS IN DEEP LEARNING

- Important types of neural networks that form the basis for most pre-trained models in deep learning:

- Artificial Neural Networks (ANN)
- Convolution Neural Networks (CNN)
- Recurrent Neural Networks (RNN)
- Generative Adversarial Networks (GAN)
- Self-Organizing Maps (SOM)

A single perceptron (or neuron) can be imagined as a Logistic Regression. Artificial Neural Network, or ANN, is a group of multiple perceptrons/ neurons at each layer. ANN is also known as a Feed-Forward Neural network because inputs are processed only in the forward direction



ANN consists of 3 layers – **Input**, **Hidden** and **Output**. The input layer accepts the inputs, the hidden layer processes the inputs, and the output layer produces the result. Essentially, each layer tries to learn certain weights.

113

113

Neural Network Topologies

There are several topologies of artificial neural networks with specific characteristics:

- networks have only forward connections (**feed-forward networks**)
- if there are feedback loops (**recurrent networks**).
- have a supervisor or teacher, that is, if they are supervised or if they have autonomous learning algorithms(**self-organized**).

114

114

Neural Networks Topologies

There are many different types of neural networks, each with its own strengths and weaknesses, and each suited for different types of tasks. **Some of the most commonly used types of neural networks:**

- **Feedforward neural networks (FNN):** Also known as multi-layer perceptrons (MLP), these are the most basic type of neural networks. They consist of an input layer, one or more hidden layers, and an output layer. The neurons in one layer are connected to the neurons in the next layer, and the information flows in one direction, from the input layer to the output layer. FNNs are used for tasks such as image classification, speech recognition, and language modeling.
- **Convolutional neural networks (CNN):** These are neural networks that are specifically designed for image and video processing tasks. CNNs consist of multiple layers of convolutional and pooling layers, which are designed to extract features from images and reduce their dimensionality. CNNs are used for tasks such as image classification, object detection, and semantic segmentation.
- **Recurrent neural networks (RNN):** These are neural networks that are designed to process sequences of data, such as time series or natural language text. RNNs consist of multiple layers of neurons, each of which is connected to the previous layer in a time-dependent manner. This allows RNNs to maintain an internal state, which allows them to process sequences of data. RNNs are used for tasks such as speech recognition, language translation, and text generation.
- **Generative Adversarial Networks (GAN):** These are neural networks that consist of two parts: a generator and a discriminator. The generator is trained to generate new data samples that are similar to the training data, while the discriminator is trained to distinguish the generated samples from the real samples. GANs are used for tasks such as image synthesis, image-to-image translation and data augmentation.
- **Self-Organizing Maps (SOM):** These are neural networks that are used for unsupervised learning tasks, such as dimensionality reduction and clustering. SOMs consist of a grid of neurons, each of which is trained to represent a specific part of the input space. SOMs are used for tasks such as visualizing high-dimensional data, anomaly detection and feature extraction.

115

115

Feedforward neural networks (FNN)

Feedforward neural networks (FNN), also known as Multi-Layer Perceptrons (MLP), are the most basic type of neural networks. They consist of an input layer, one or more hidden layers, and an output layer. The neurons in one layer are connected to the neurons in the next layer, and the information flows in one direction, from the input layer to the output layer. The input layer receives the input data, and the output layer produces the output. The hidden layers are used to extract features from the input data and to make predictions or decisions.

FNNs are used for a wide range of tasks, such as image classification, speech recognition, and language modeling. They can be trained using supervised learning algorithms, such as backpropagation, to learn the relationship between the input and output data. The training process involves adjusting the weights and biases of the neurons in the network, so that the output of the network matches the desired output.

One of the main advantages of FNNs is their simplicity, which makes them easy to understand and implement. They can also be used to model non-linear relationships between the input and output data. However, their performance might be limited when the input data has high dimensionality or when the relationship between the input and output data is complex.

116

116

Applications

- **Health care:** With easier access to accelerated GPU and the availability of huge amounts of data, health care use cases have been a perfect fit for applying DL.
- **Autonomous vehicles:** Though self-driving cars is a risky field to automate, it has recently taken a turn towards becoming a reality. From recognizing a stop sign to seeing a pedestrian on the road, deep learning-based models are trained and tried under simulated environments to monitor progress.
- **e-commerce:** Product recommendations has been one of the most popular and profitable applications. With more personalized and accurate recommendations, customers are able to easily shop for the items they are looking for and are able to view all of the options that they can choose from. This also accelerates sales and thus, benefits sellers.
- **Personal assistant:** Thanks to advancements in the field of DL, having a personal assistant is as simple as buying a device like Alexa or Google Assistant. These smart assistants use DL in several aspects such as personalized voice and accent recognition, personalized recommendations, and text generation.

Stock market predictions and weather predictions are also popular fields in which deep learning has been helpful.